Smart Parking System — Programmer Manual

Version	`smart_parking_v9.ino` + `parking_page.h` (measured 1:100 model, October 2026)
Target	ESP32-WROOM32 (CH9102 USB bridge), Arduino framework, ESP32 core 3.0.1
Date	18/10/2026
Authors	Aseel Kurdy, Nayef Habiballah, Osama Abdulhalim

> The code comments are in Hebrew; this manual is in English so that code identifiers stay readable.

1. Build & toolchain

IDE	Arduino IDE 2.x
Board package	`esp32` by Espressif (3.0.1)
Board selection	**ESP32 Dev Module**
Serial baud	9600
Libraries	`WiFi.h`, `WebServer.h` — both ship with the ESP32 core

1.1 File layout

```
smart_parking_v9/
├── smart_parking_v9.ino     <- firmware
└── parking_page.h           <- web page (HTML/CSS/JS) as a PROGMEM string
smart_parking_v9_research/   <- identical, but distances are reported as real-scale metres
tests/                       <- simulator, test suite, fuzz, ESP32 compile check (section 13)
docs/                        <- builds the PDF manuals
```

`parking_page.h` **must** sit in the sketch folder; it is included only when `ENABLE_WEB_VIEW` is 1.

1.2 Sketch sections

Section	Content
`[1]`	Pins, sensor calibration, timing
`[1ב]`	Web-view configuration
`[2]`	Measured dimensions of the model
`[3]`	Spot table and car bookkeeping
`[4]`	Road network and A*
`[4ב]`	Spot state, text picture, JSON builder
`[5]`	Lot operations (entry, park, leave, reroute, timeout, sensor events)
`[6]`	Command parser
`[7]`	Sensor reading and entrance classification
`[7ב]`	HTTP server
`[8]`	`setup()` / `loop()`

2. Configuration constants

All tunables are at the top of the sketch. Changing them requires a re-upload.

2.1 Hardware pins — section [1]

```
const int trigPin1 = 27, echoPin1 = 14;   // entrance sensor (side-facing)
const int trigPin2 = 26, echoPin2 = 25;   // spot-0 sensor (top-down)
const int ledPin   = 2;                   // on-board blue LED
```

2.2 Sensor calibration — section [1]

```
// entrance sensor: what it reads on the model
const float GATE_EMPTY_CM = 30.0;   // empty road = the wall opposite (no echo = empty too)
const float GATE_ENTRY_CM = 22.0;   // a car entering
const float GATE_EXIT_CM  = 0.0;    // optional separate exit lane seen by the same sensor; 0 = none
const int   GATE_CONFIRM_SAMPLES = 2;
const int   GATE_CLEAR_SAMPLES   = 2;

// spot 0 (top-down): empty 25-26 cm, car 22.5-23.5 cm
const float SPOT_ARRIVE_CM = 24.0;  // below -> a car arrives
const float SPOT_LEAVE_CM  = 24.5;  // above -> the car left
const float MIN_VALID_CM   = 2.0;   // shorter readings are electrical noise
```

- **Entrance:** you give the *measured* readings, not a threshold. Each reading is assigned to the nearest of `GATE_EMPTY_CM`, `GATE_ENTRY_CM` (and `GATE_EXIT_CM` if non-zero), so the threshold is the midpoint: a car is counted below **26.0 cm**. `printGateBands()` prints the result at boot.
- **Spot 0:** two thresholds (hysteresis). They must satisfy `car readings < SPOT_ARRIVE_CM < SPOT_LEAVE_CM < empty readings`. With the measured 22.5–23.5 / 25–26 cm this leaves about 0.5 cm on each side — enough, but tight. Getting the order wrong latches the sensor in one state (a real bug in an early version).
- **Optional exit lane:** with `GATE_EXIT_CM > 0` (e.g. exit 22, entry 26), a car in the exit lane removes a car from the lot (`onGateExit()`), see section 6.3.

2.3 Timing — section [1]

Constant	Value	Effect
`SAMPLE_INTERVAL_MS`	500	Sensor sampling period
`SPOT_CONFIRM_SAMPLES`	4	Agreeing samples for a spot-0 change (≈ 2 s)
`GATE_CONFIRM_SAMPLES` / `GATE_CLEAR_SAMPLES`	2 / 2	Entrance: samples to count a car / to re-arm (≈ 1.1 s)
`CROSSTALK_DELAY_MS`	60	Gap between the two sensor reads (HC-SR04 datasheet)
`CHECK_DEFAULT_S`	3	Default duration of `check`
`BLINK_INTERVAL_MS`	300	LED blink half-period
`RESERVATION_TIMEOUT_MS`	60000	Reservation lifetime
`EXIT_MATCH_MS`	20000	Exit lane only: a car that just left spot 0 is the one seen in the exit lane

2.4 Web view — section `[1ב]`

```
#define ENABLE_WEB_VIEW 1          // 0 = no WiFi; the Serial Monitor draws the text picture instead
const char* WEB_SSID = "SmartParking";
const bool  TEXT_MAP = false;      // true = also draw the text picture while the page runs
```

`showTextMap()` returns `TEXT_MAP || !webRunning`, so the text picture is an automatic fallback.

2.5 Measured model — section `[2]`

```
const float ENT_TO_FLOOR0_CM   = 22.0;  // entrance -> floor-0 lane, just before spot 0
const float ENT_TO_RAMP_CM     = 17.5;  // entrance -> start of the ramp
const float RAMP_CM            = 20.0;  // ramp start -> floor-1 entrance
const float FLOOR1_TO_FIRST_CM = 7.5;   // floor-1 entrance -> floor-1 lane, just before spot 7
const float SPOT_WIDTH_CM      = 2.5;   // lane step between neighbouring stalls
const float SPOT_LENGTH_CM     = 5.0;   // lane -> inside the stall
const float CAR_LENGTH_CM = 4.0, CAR_WIDTH_CM = 2.0, CAR_HEIGHT_CM = 2.5;
const char* DIST_UNIT = "cm";           // "m" in the research variant (1:100)
const int   FLOORS = 2, SPOTS_PER_FLOOR = 7;
```

Resulting distances: floor 0 = 27.0, 29.5 … 42.0 cm; floor 1 = 50.0, 52.5 … 65.0 cm. Every constant is reported in the JSON (`geo`) so the page shows the same numbers.

---

3. The lot model

3.1 Why a road network

On the measured model the ramp starts *before* the floor-0 stalls (17.5 cm vs 22 cm) and the first road segment is much longer than the stall pitch. The earlier fixed-step grid cannot represent this, so the lot is a small graph whose edges carry the measured lengths.

3.2 Nodes and edges — section `[4]`

Node index	Meaning
`NODE_ENT` (0)	main entrance
`NODE_RAMP` (1)	start of the ramp
`NODE_FLOOR1` (2)	floor-1 entrance (top of the ramp)
`NODE_LANE0 + i` (3–16)	point in the lane in front of spot `i`
`NODE_SPOT0 + i` (17–30)	inside spot `i`

`buildRoadNetwork()` creates the edges:

```
addEdge(NODE_ENT,    NODE_LANE0,                  ENT_TO_FLOOR0_CM);
addEdge(NODE_ENT,    NODE_RAMP,                   ENT_TO_RAMP_CM);
addEdge(NODE_RAMP,   NODE_FLOOR1,                 RAMP_CM);
addEdge(NODE_FLOOR1, NODE_LANE0 + SPOTS_PER_FLOOR, FLOOR1_TO_FIRST_CM);
for each spot i:
  if i is not the last on its floor: addEdge(NODE_LANE0+i, NODE_LANE0+i+1, SPOT_WIDTH_CM);
  addEdge(NODE_LANE0+i, NODE_SPOT0+i, SPOT_LENGTH_CM);
```

30 edges, 31 nodes. Edges are undirected.

3.3 Spot table — section `[3]`

```
const int TOTAL_SPOTS = 14;
const uint8_t SPOT_FL[TOTAL_SPOTS] = {0,0,0,0,0,0,0, 1,1,1,1,1,1,1};
int spotIndexOnFloor(int i) { return i % SPOTS_PER_FLOOR; }   // 0 = first from the floor's entrance
```

Spots are numbered by increasing distance from the entrance (spot 0 nearest). Parallel arrays rather than a `struct` : **the Arduino IDE inserts auto-generated prototypes before user types are declared**, so a `struct` in a function signature fails to compile.

---

4. State variables

4.1 Per-spot arrays — section `[3]`

Variable	Type	Meaning
`spotDist[14]`	`float`	A* distance from the entrance (cm), computed once in `setup()`
`occupied[14]`	`bool`	A car is physically parked here
`assignedCarId[14]`	`int`	`>= 0` → reserved for that car; `-1` → not reserved
`reservedAtMs[14]`	`unsigned long`	`millis()` when the reservation was created
`parkedCarId[14]`	`int`	Which car is parked here
`wasCorrect[14]`	`bool`	Whether the parked car was in its assigned spot

4.2 Global state

Variable	Meaning
`nextCarId`	Next ID to hand out; the first car gets **0**
`carExited[500]`	Whether car `i` has left the lot
`spotSensedStable` , `spotDisagreeCount`	Debounced spot-0 state and the run of contradicting samples
`gateLatched` , `gateCand` , `gateCandCount` , `gateClearCount`	Entrance state machine (section 8.3)
`spotLeavePending` , `spotLeaveAtMs`	Exit lane only: the car that just left spot 0
`firstSample`	Suppresses events on the very first sample
`edgeA/edgeB/edgeLen/edgeCount`	The road network
`statusSeq`	Incremented by `status` ; the browser watches it
`webRunning`	True only after the AP and the server started

Memory: under 3 KB in total, negligible against the ESP32's ~320 KB of SRAM.

5. A* — section `[4]`

5.1 `float aStarDistance(int goalSpot)`

Shortest driving distance (cm) from `NODE_ENT` to `NODE_SPOT0 + goalSpot`, or `-1`. The open set is a linear scan over the 31 nodes; neighbours come from a scan over the 30 edges. A spot node other than the goal is never expanded — **a car cannot drive through a stall**.

5.2 Heuristic

```
float heuristic(int node, int goalSpot) {
  if (node == NODE_SPOT0 + goalSpot) return 0;
  if (node is a lane node on the goal's floor)
    return |index(node) - index(goal)| * SPOT_WIDTH_CM + SPOT_LENGTH_CM;
  return SPOT_LENGTH_CM;
}
```

Admissible: every route ends with the `SPOT_LENGTH_CM` edge into the stall, and from a lane node on the same floor the car must also drive along the lane to the stall. h never overestimates, so A* returns the shortest path. (No closed set; a node can be re-opened. At 31 nodes this costs nothing.)

5.3 `void computeSpotDistances()`

Called once from `setup()`: builds the network and runs A* for all 14 spots into `spotDist[]`. **No search happens at runtime.** A single Dijkstra from the entrance would give every distance in one pass; A* was the specified algorithm.

---

6. Lot logic — section `[5]`

6.1 Query helpers

Function	Returns
`int findAvailableSpot(float* outDist)`	Nearest spot not occupied, not reserved and not blocked; `-1` if none
`int whereIsCarParked(int carId)` / `whereIsCarReserved(int carId)`	Spot index, else `-1`
`bool isCarInLot(int carId)`	Car exists and has not exited
`int newestUnparkedCar()`	Highest-numbered car still in the lot that has not parked
`bool spot0Blocked()`	Sensor sees an object at spot 0 but no car is registered there
`int collectWaitingCars(int* ids, int maxIds)`	Cars in the lot that have not parked (total; first `maxIds` stored)

6.2 Operations

- `int assignNewCar()` — refuses with `LOT FULL` before allocating an ID (a full lot does not burn IDs), registers the car, reserves the nearest spot, prints `[ENTRY]` and the next-car recommendation.
- `void parkCarAt(int spot, int carId)` — validates (spot range, id, car in lot, not already parked, spot not occupied), then parks. Parking in a spot reserved for another car is **accepted** (a barrier cannot stop a car); the displaced car is rerouted, or leaves if the lot is full.
- `void leaveCar(int carId, bool fromSensor)` — the car leaves the lot, parked or still driving.
- `void checkReservationTimeouts()` — every `loop()`: releases reservations older than 60 s; the car stays in the lot.

6.3 Sensor events

Function	When	What
`onSpotSensorArrive(cm)`	spot 0 confirmed occupied	parks the car the spot is reserved for, else `newestUnparkedCar()`, else marks spot 0 `BLOCKED`
`onSpotSensorLeave(cm)`	spot 0 confirmed empty	the car registered there leaves the lot
`assignNewCar()`	entrance: car confirmed	same as the `new` command
`onGateExit(cm)`	exit lane only (`GATE_EXIT_CM > 0`)	removes a car: the one that just left spot 0 is matched (`EXIT_MATCH_MS`), else `pickCarForExit()` = car at spot 0, else lowest parked id, else any car in the lot

---

7. Spot state — section `[4]`

```
const int SP_FREE = 0, SP_RESERVED = 1, SP_OCCUPIED = 2, SP_BLOCKED = 3;
int spotState(int i);   // occupied > reserved > blocked (spot 0 only) > free
```

---

8. Sensor layer — section `[7]`

8.1 `float readDistanceCm(int trigPin, int echoPin)`

10 µs trigger, `pulseIn()` with a 30 ms timeout; returns `-1` when no echo returns. `(duration / 2.0) / 29.1` converts µs to cm.

8.2 `bool validReading(float cm)`

`cm >= MIN_VALID_CM`. Shorter pulses are noise on the Echo line (measured 0.0–0.1 cm).

8.3 Entrance — `int classifyGate(float cm)` and the state machine in `loop()`

`classifyGate()` returns `GATE_EMPTY`, `GATE_ENTRY` or `GATE_EXIT` — the nearest configured reading; no echo or anything at/after the wall is `GATE_EMPTY`; noise (< 2 cm) is ignored entirely. A car is counted after

`GATE_CONFIRM_SAMPLES` equal readings in a row and only once (`gateLatched`) until the road reads empty `GATE_CLEAR_SAMPLES` times.

8.4 Spot 0 — debounce in `loop()`

```
bool spotSensed = spotValid && distSpot < (spotSensedStable ? SPOT_LEAVE_CM : SPOT_ARRIVE_CM);
if (!spotValid || spotSensed == spotSensedStable) spotDisagreeCount = 0;
else if (++spotDisagreeCount >= SPOT_CONFIRM_SAMPLES) { /* change state, fire the event */ }
```

The active threshold depends on the current state (hysteresis); an invalid reading resets the count, because noise arrives in bursts.

---

9. Command parser — section `[6]`

`Serial -> lineBuf (48) -> handleCommand() -> splitTokens() -> parseNumber() -> operation`

Command	Args	Handler
`new`	0	`assignNewCar()`
`park`	2	`parkCarAt(spot, id)`
`leave` / `cancel`	1	`leaveCar(id, false)`
`status`	0	`printStatus()`
`check`	0–1	sets `checkUntilMs`
`help`	0	`printHelp()`

Commands are case-insensitive (`strcasecmp`), argument counts are exact, `parseNumber()` (`strtol`) rejects trailing junk and huge values, and a token longer than 15 characters is stored empty instead of being truncated into a different valid value.

---

10. Web layer

10.1 Server — section `[7ב]`

```
WebServer webServer(80);
webServer.on("/",      handleWebRoot);    // PARKING_PAGE from PROGMEM
webServer.on("/state", handleWebState);   // buildStateJson()
```

`startWebView()` starts an open soft AP named `SmartParking` (no internet). On failure `webRunning` stays false and the Serial Monitor draws the text picture.

10.2 `String buildStateJson()` — the payload

```
{"up":1,"seq":0,"ttl":60,"floors":2,"per":7,"unit":"cm",
 "geo":{"e0":22.0,"er":17.5,"ramp":20.0,"f1":7.5,"sw":2.5,"sl":5.0,"car":[4.0,2.0,2.5]},
 "real":0,"next":0,"nextd":27.0,
 "sp":[{"f":0,"k":0,"s":0,"car":-1,"ok":1,"left":0,"d":27.0}, ...],
 "waitN":0,"wait":[]}
```

Key	Meaning
`up`, `seq`, `ttl`	uptime (s), `status` counter, reservation lifetime (s)
`floors`, `per`, `unit`	2 floors, 7 spots per floor, distance unit
`geo`	measured lengths: entrance→floor-0 lane, entrance→ramp, ramp, floor-1→lane, stall width/length, car L/W/H
`real`	index of the physical spot
`next`, `nextd`	spot for the next car (`-1` = full) and its distance
`sp[i]`	`f` floor, `k` position on its floor, `s` state, `car`, `ok`, `left` seconds, `d` distance
`waitN`, `wait[k]`	cars still driving (total) and up to 16 of them: `car`, `spot` (`-1` = expired), `left`

Size: about 1.5 KB in the worst case.

10.3 Browser side — `parking_page.h`

One PROGMEM raw string with HTML, CSS and JavaScript. **No CDN and no external fonts** — the AP has no internet.

The model is drawn in centimetres (`viewBox="0 0 58 48"`), traced from the photo of the model (`M` holds where things are). Stall and car sizes come from `geo`, the distances shown are the measured ones from the board. Three stacked SVG layers — base (floor 0, road, entrance, sensors), ramp, and deck (floor 1 with the U-turn) — plus an overlay for routes; in 3D the board is tilted with CSS, the deck layer is lifted (`translateZ`) and the ramp layer tilted so it meets it.

Function	Role
`poll()`	`fetch('/state')` every second, 2.5 s abort
`draw(s)`	counters, banner, side cards; redraws stalls/routes when the state changes; starts arrival animations
`buildBoard()`	draws the model once (grass, roads, floors, ramp, U-turn, deck, sensors)
`stall(p)`, `carAt(r)`, `route(i)`	where a stall is, where a car stands in it, the driving path to it
`drawSpots(s)`	state of every stall (free / next / reserved / parked right or wrong / blocked)
`drawRoutes(s)`	dashed route of the next car, orange route of the selected stall
`animateArrival(car, spot)`, `stepMovers()`	a new car drives from the entrance to its stall
`drawWaiting`, `drawInfo`, `drawDimsTable`, `drawDims`	side cards and the dimension labels
`setView(v)`	2D / 3D (`?view=3d` opens in 3D)

11. Main loop — section `[8]`

```
void loop() {
  if (webRunning) webServer.handleClient();   // 1. HTTP
  while (Serial.available()) { ... }          // 2. commands
  checkReservationTimeouts();                 // 3. expiries
  if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) { ... }   // 4. entrance + spot 0 sensors
  // 5. LED: off = no car, solid = right car, blinking = wrong car
}
```

Blocking: `CROSSTALK_DELAY_MS` (60 ms) and up to 30 ms per `pulseIn()`. Printing the text picture at 9600 baud blocks for about 1.4 s, which is why it is only a fallback.

12. How to modify common things

1. **Distances:** edit the constants of section `[2]`. A*, the JSON, the page labels and the tests follow automatically.
2. **Sensor calibration:** run `check 10` empty and with a car; put the readings in `GATE_EMPTY_CM` / `GATE_ENTRY_CM`; for spot 0 keep `car < SPOT_ARRIVE_CM < SPOT_LEAVE_CM < empty`.
3. **Exit lane:** set `GATE_EXIT_CM` to the exit-lane reading (and `GATE_ENTRY_CM` to the entry-lane reading).
4. **Physical spot:** change `REAL_SPOT_INDEX`; sensor handlers, LED and `[PHYSICAL]` follow.
5. **Number of spots / layout:** `TOTAL_SPOTS`, `SPOTS_PER_FLOOR`, `SPOT_FL`, `buildRoadNetwork()`, and the drawing positions in `M` / `stall()` in the page.
6. **Disable WiFi:** `#define ENABLE_WEB_VIEW 0`.

7. **RFID instead of the ID counter:** replace the attribution in `onSpotSensorArrive()` with a UID lookup; `parkCarAt()` already takes an explicit `carId`.

13. Testing (folder `tests/`)

Tool	What it does
`host_sim/`	The **unmodified** `.ino` compiled on the PC with stand-ins for Serial, the clock, both HC-SR04 sensors (with noise models), the LED and the web handlers. `build.ps1` builds variants: text picture, as shipped, WiFi off, exit lane.
`run_tests.ps1 -Sim`	The 38 original hardware tests plus `SIM-*` (sensors, LED) and `VIS-*` (JSON, picture). `-Slow`, `-Soak`, `-Only`. With `-Port COMx` it drives the real board (set `TEXT_MAP = true` first).
`fuzz_model.py`	Independent Python model of the rules; after every random operation the firmware's JSON must equal the model.
`edge_cases.py`	3-digit car IDs, more than 16 waiting cars, worst-case payload size.
`esp32_syntax_check.ps1`	Compiles the sketch with the real Xtensa compiler and ESP32 core headers (syntax/type check, no link).

14. Pitfalls

Pitfall	Why it matters
`struct` in a function signature	Arduino's auto-prototypes come before the type → compile error
Spot thresholds in the wrong order	The sensor latches and never fires a leave event
The wall at the entrance closer than the midpoint	Every reading counts as a car (ghost cars)
`atoi` on user input	Maps `"abc"` to 0; use `parseNumber()`
Hebrew in Serial output	The Serial Monitor has no RTL support; all runtime output is English
`millis()` rollover (~49 days)	Subtraction-based comparisons are safe; `checkUntilMs` uses a direct comparison
Removing `CROSSTALK_DELAY_MS`	One sensor picks up the other's echo

15. Function index — `smart_parking_v9.ino`

Function	Section	Purpose
`spotIndexOnFloor`	`[3]`	Position of a spot on its floor
`addEdge`, `buildRoadNetwork`	`[4]`	Build the road network
`heuristic`, `aStarDistance`, `computeSpotDistances`	`[4]`	A* and the distance table
`spot0Blocked`, `findAvailableSpot`, `whereIsCarParked`, `whereIsCarReserved`, `clearReservationFor`, `isCarInLot`, `newestUnparkedCar`	`[4]`	Queries
`spotState`, `reservationSecondsLeft`, `collectWaitingCars`	`[4↵]`	State
`printSpotCell`, `printMapRow`, `printMapLine`, `printWaitingLine`, `printLotMap`	`[4↵]`	Text picture
`buildStateJson`	`[4↵]`	JSON for the page
`registerCar`, `reserveSpotFor`, `printSpotAndDist`, `printNextRecommendation`, `printLotUpdate`	`[5]`	Helpers
`assignNewCar`, `rerouteCar`, `parkCarAt`, `leaveCar`, `checkReservationTimeouts`	`[5]`	Operations
`onSpotSensorArrive`, `onSpotSensorLeave`, `pickCarForExit`, `onGateExit`	`[5]`	Sensor events
`printStatus`, `printHelp`	`[5]`	Output
`splitTokens`, `parseNumber`, `argsOk`, `handleCommand`	`[6]`	Parser
`readDistanceCm`, `validReading`, `classifyGate`, `printGateBands`	`[7]`	Sensors
`handleWebRoot`, `handleWebState`, `startWebView`	`[7↵]`	HTTP